From Prompt to Kinematics: Self-Verifying Generation of Valid, Controllable Robot Models

***Abstract*—Turning a natural-language description into a robot model that actually runs in a simulator remains slow and expert-driven: authors hand-write MJCF or URDF, then debug models that compile but are physically implausible—interpenetrating at rest, diverging under control, or missing parts the prompt asked for. We present Prompt2Kinematics, an agentic system that authors valid, controllable MuJoCo models from a prompt (optionally conditioned on a reference image) inside a self-verifying loop. The agent writes MJCF through a constrained tool interface and, after every edit, invokes a verifier that compiles the model and step-probes it for instability; the same loop covers agent-authored control panels and custom CAD parts. Generated models are inspected and edited in a live physics viewer through grounded direct manipulation. We evaluate the system with an independent scorer (invalidity rate, pass@1 over compile, stability, controllability, and requirement compliance) and a vision-based judge of geometric and pose correctness that physics checks cannot capture. Across a 25-prompt benchmark (one generation per prompt), the full loop achieves pass@1 $= 0.92$ at an invalidity rate of 0.00, with every model stable at rest and controllable. A matched ablation over the same prompts raises pass@1 from 0.32 for a direct use of the same LLM to 0.64 with our domain prompt and tools but no verification, and to 0.92 with the loop enabled, lifting the stable rate from 0.72 to 1.00.**

***Index Terms*—robot design, simulation, large language models, program synthesis, MuJoCo, self-verification, human-robot interaction**

I. Introduction

Physics simulators such as MuJoCo [1] are central to modern robotics, but getting a usable model *into* the simulator is a bottleneck. A model is written by hand in MJCF or URDF, XML-based formats that couple kinematics, inertial properties, actuators, collision geometry, and solver settings, and small mistakes are punishing: a model can parse and load yet interpenetrate at rest, diverge the instant an actuator is driven, or silently omit a component the designer intended. Authoring is therefore an expert, iterative task, which limits who can prototype a robot and how quickly ideas can be tried.

Large language models can generate structured description formats, and recent work turns text into articulated objects or robot morphologies. Yet a generated XML string that compiles is not the same as a model that is *physically valid* and *controllable*. Our results reinforce this distinction: all three evaluated conditions achieve a zero compilation-invalidity rate, yet their physical stability differs substantially, from 0.40 for direct LLM generation to 0.72 without verification and 1.00 with the full self-verifying loop. The core difficulty is a gap between what a designer means and what the artifact does: the prompt expresses *intent* ("a four-wheeled rover I can drive"), while the failure modes, such as an unstable rest pose, an under-damped servo, or a wheel with collision disabled, are *dynamic and geometric* properties that only surface when the model is stepped or looked at. Closing this intent–expression gap requires more than one-shot generation; it requires the system to *verify* its own output against the physics and to give the designer a direct way to inspect and correct it. Prompt2Kinematics addresses this with three ideas. First, generation runs inside a *self-verifying loop*: the agent authors MJCF through a small, sandboxed

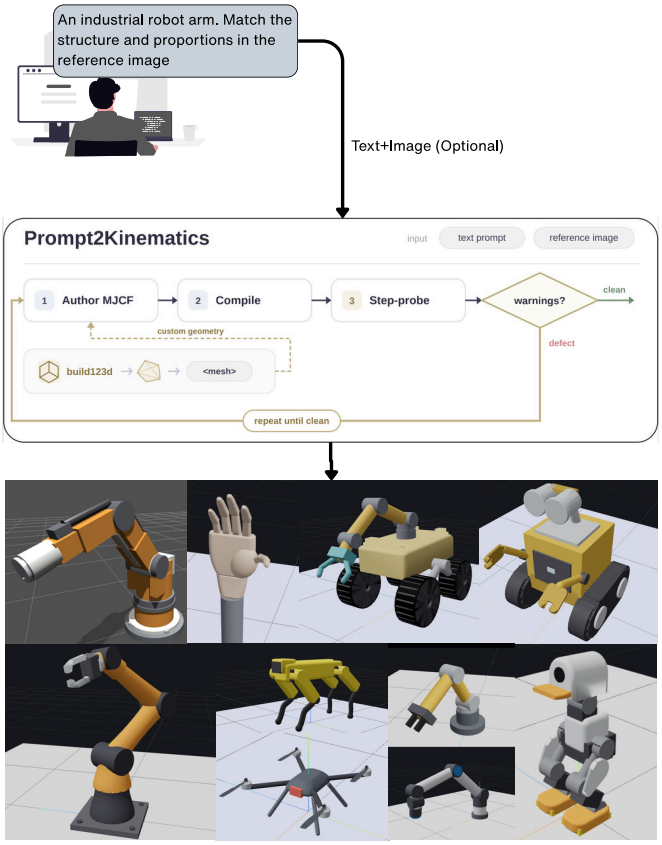


Fig. 1. Prompt2Kinematics authors MJCF from language and imagery, then verifies it in simulation before returning it. Every robot shown is system output.

tool interface, and after every edit a verifier compiles the model and step-probes it for instability (deep interpenetration, runaway velocity at rest), returning warnings the agent must resolve before finishing. The same discipline covers control-panel specifications and custom CAD parts. Second, the system supports *image-conditioned* authoring: a reference photograph guides structure and proportions, and the model can later be judged against that same image. Third, generated models are not static outputs but live simulations the designer edits through *grounded direct manipulation*: selecting and repositioning parts in a running physics view, with edits reconciled against the model.

We contribute: (i) a system architecture that couples an LLM agent to a physics simulator through a constrained, self-verifying tool loop; (ii) an image-conditioned authoring path and a vision-based judge that scores geometric and pose correctness (uprightness, symmetry, part orientation, reference match) beyond what physics checks catch; and (iii) an evaluation methodology, consisting of an independent scorer and a loop ablation, quantifying how much the self-verification loop contributes to producing valid, controllable models.

On a 25-prompt benchmark spanning serial arms, parallel and Cartesian mechanisms, grippers and hands, wheeled and legged bases, a serpentine chain, an aerial vehicle, and image-conditioned targets, the system returns a model that compiles for every prompt (invalidity rate 0.00) and that is stable at rest and controllable in all 25 cases; 23 of the 25 also satisfy their structural requirement specification, giving pass@1 $= 0.92$. Both failures violate a structural requirement rather than a physical one, so validity is never the binding constraint. How much of this is attributable to verification rather than to the underlying model is isolated by the ablation in Section V-B: a direct use of the same LLM reaches pass@1 $= 0.32$ on these prompts, our agent without any verification signal 0.64, and the full loop 0.92.

II. Related Work

A. Static Geometry to Articulated Objects

Recent work has gradually moved from generating static geometry toward generating articulated and executable objects. Text2CAD [2] converts natural-language descriptions into parametric CAD construction sequences, showing that structured geometric models can be generated directly from text. Later work goes beyond static shape and models how parts move relative to each other. Articulate Anything [3] generates articulated assets from multimodal input and iteratively improves them using visual feedback. URDF-Anything [4] similarly reconstructs articulated objects and their kinematic structure in a robot-description format.

Together, these systems show that foundation models can reason about not only shape, but also parts, joints, and articulation. Their main focus, however, is articulated asset generation or reconstruction. Prompt2Kinematics addresses a different problem: the user describes a desired robot, optionally with a reference image, and the system authors the robot model itself while checking the resulting model in the target physics simulator.

B. Language-Guided Robot Design

A related line of work uses language models to generate or optimize the robot embodiment itself. Text2Robot [5] maps text descriptions to quadruped morphologies and controllers, with an emphasis on physically realizable designs. RoboMorph [6] places an LLM inside an evolutionary design loop, where candidate robot morphologies are generated, trained, evaluated on task performance, and refined over multiple generations.

These systems show that language models can be useful in robot-design pipelines, but their main goal is usually to search for or optimize a morphology according to task performance, manufacturability, or another design objective. Prompt2Kinematics instead focuses on realizing a robot that the user has already described. The goal is not to search for the best-performing morphology, but to produce an editable MuJoCo model that matches the requested structure and remains valid in simulation.

C. Task and Simulation Environment Generation

Generative models have also been used to automate the parts of simulation that surround the robot. GenSim [7] generates robotic simulation tasks and demonstrations, while RoboGen [8] automatically creates tasks, scenes, training supervision, and learning procedures. More recently, SR-Platform [9] generates executable MuJoCo environments from natural-language descriptions through a staged pipeline for scene planning, asset generation, spatial layout, and final MJCF assembly. These approaches reduce the effort needed to build tasks and environments for robot learning. Prompt2Kinematics works at a complementary level: its main generated artifact is the articulated robot itself, including its bodies, joints, actuators, collision geometry, custom parts, and controls, rather than the task or scene around an existing robot.

D. Execution Feedback and Interactive Authoring

Across these areas, iterative feedback has become an important way to improve generated outputs. Self-debugging approaches [10] and Reflexion [11] show that generated programs or agent behavior can improve when failures from earlier attempts are used as feedback. Related robotics systems discussed above also use iterative feedback: Articulate Anything [3] uses visual feedback to refine articulated models, while RoboMorph [6] uses downstream task performance to guide later morphology changes.

Prompt2Kinematics applies this idea directly to robot authoring by using the target physics simulator as part of the generation loop. After model edits, MuJoCo compiles and steps the generated MJCF. Compilation failures and rest-pose instability signals are returned to the agent so that it can repair the model before finishing. The final artifact is then evaluated

separately for properties such as stability, controllability, and structural requirement compliance.

The system also keeps the designer involved in the authoring process. Following the direct-manipulation principle [12], the generated robot remains a live simulation that the user can inspect, actuate, select, and reposition. These interactions can also provide context for later natural-language edits. In this way, Prompt2Kinematics combines generative robot modeling, execution-grounded repair, and interactive authoring in a single workflow.

III. System Design

A. Overview

A session is an isolated workspace holding robot models (`robots/<name>/<name>.xml` with mesh siblings), custom parts (`parts/<name>/`), and control-panel specifications. A prompt is handled by an LLM agent that operates only through a fixed set of tools; all file access is confined to the session folder. The agent authors and edits MJCF, validates it, optionally builds and mounts CAD parts, and writes a control panel. The resulting model is streamed to a browser viewer that runs the same physics engine, where the designer plays, inspects, and directly manipulates it.

B. Constrained agent and tool gate

The agent is given file read/write/edit tools plus a small set of domain tools: `validate` (compile and stability-probe a model), `validate_controls` (check a control panel against the model's actuators), `build_part` (compile a CAD script to a mesh and structurally check it), `use_part` (mount a part on a robot), and `ask_user` (request a decision when the prompt is materially ambiguous). Every tool call passes through an authoritative gate that enforces an allowlist and confines all paths to the session folder, so the agent cannot run a shell or escape its workspace. This constrained interface is what makes the loop safe to run unattended over a benchmark.

C. Self-verifying generation loop

The central mechanism is verification after every edit. `validate` compiles the MJCF and then *steps* it under zero control from the rest pose, flagging two instability signatures: deep interpenetration (a static overlap beyond a small tolerance) and runaway velocity (joint speeds exceeding a threshold over a fixed horizon at zero control). Crucially, MuJoCo can integrate a diverging model to non-finite state *without raising an error*; the step-probe is what surfaces the failure. Warnings are returned to the agent as first-class errors to be fixed—most often by disabling self-collision among a robot's own overlapping link geoms while keeping collision only where the robot must contact the world (feet, wheels). The same loop covers control panels (validated against actuator names and ranges) and CAD parts (built and structurally checked before mounting). The system prompt additionally encodes physical-realism rules learned to matter in practice: an implicit integrator for stiff position servos, damping/armature on actuated joints, a floating base for mobile robots, and building every named component at the requested count.

D. Custom CAD parts

When a prompt needs geometry that primitives cannot express, the agent authors a build123d B-rep script (`parts/<name>/part.py`) in millimetres with its origin at the intended mount point, and `build_part` runs it in a sandboxed subprocess to produce an STL, checking it for watertightness, degeneracy, and suspicious size. `use_part` copies the mesh into a robot and the agent references it as a mesh geom; the assembled model is then re-validated. This lets a prompt like "a bipedal duck robot" yield custom body, head, beak, and foot meshes that still satisfy the stability probe once assembled. Because an agent left to itself tends to prefer primitive geometry—a primitive model is short and almost always compiles, whereas a CAD part has many more ways to fail—the authoring prompt is written to make the agent decide when a shape warrants custom geometry rather than wait to be asked.

E. Image-conditioned authoring

A prompt may include a reference image (a photograph of a target robot, a diagram, or a screenshot of a defect to fix). The agent reads the image and uses it as the message directs—most importantly, matching structure and proportions when building from a visual reference. The reference is retained with the session so the model can later be judged against it (Section IV-D).

F. Grounded direct manipulation

The viewer renders the live simulation and exposes agent-authored control panels (sliders, buttons, oscillators for gaits) bound to actuators. Beyond driving the robot, the designer can *anchor* a part—select a body in the 3D view—to ground a subsequent natural-language edit, and can drag a part to reposition it, with the change reconciled into the model's local frame and re-validated before it persists. The running simulation is thus the shared object that both the agent and the designer act on.

G. Implementation

The agent runs on `claude-sonnet-4-5`; the vision form judge of Section IV-D runs on `claude-opus-5`. The judge is thus a different and stronger model than the generator, so it does not grade its own output. All three ablation conditions use the same generator model and differ only as described in Section V-B.

IV. Evaluation Methodology

A. Metrics

We score a generated model on four binary properties: it *compiles*; it is *stable* (the rest-pose step-probe raises no warning); it is *controllable* (it has at least one actuator with a finite control range and does not diverge when actuators are driven

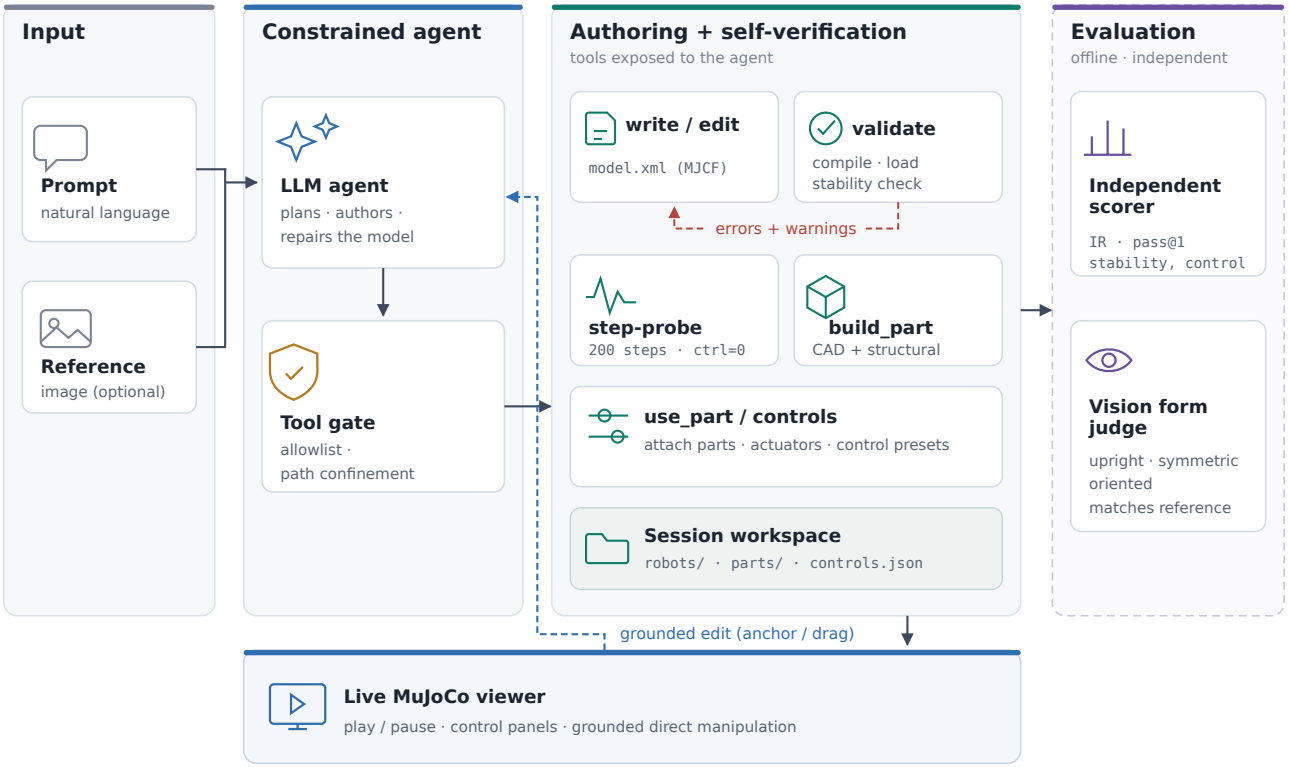


Fig. 2. System architecture. A natural-language prompt (optionally with a reference image) is handled by a constrained LLM agent whose every tool call passes through a gate (allowlist + path confinement). The agent authors MJCF and custom CAD parts through authoring and self-verification tools; `validate` (compile + stability step-probe) and `build_part` (CAD + structural check) return errors and warnings to the agent (red), closing the self-verifying loop. The resulting model in the session workspace is streamed to a live MuJoCo viewer for play, control panels, and grounded direct manipulation (blue), and is scored offline by an independent scorer and a vision form judge.

to their range extremes); and it is *requirement-compliant* (it satisfies a small machine-checkable specification of the prompt's structural demands, e.g., a floating base, a minimum number of hinge joints, a minimum number of actuators). A model *passes* if it is stable, controllable, and compliant (compilation is implied). Over a set of generations we report, following validity metrics used in Text2CAD [2] and code-generation conventions [13]: the *invalidity rate* (fraction that fail to compile), *pass@1* (fraction of all generations that pass), plus per-property rates. Scoring is performed by an *independent* module that re-compiles and re-probes the final model; the agent's own verifier is never trusted for scoring. We additionally report generation cost. For single-turn conditions, wall-clock is the interval from prompt submission to the agent's final message. For multi-turn sessions, raw session span is meaningless—it includes arbitrary user idle time, and in our logs ranges from 18 seconds to 36 days for comparable models—so we report *active generation time*, the sum of prompt to turn end intervals only. Token counts are billed totals and are dominated by prompt-cache reads; we report the cache split rather than a single figure, since billed volume at full input rate overstates cost by roughly fivefold.

B. Benchmark

The benchmark contains 25 prompts chosen to span the range of demands the system has to meet rather than to fill fixed difficulty tiers. They cover a breadth of morphologies, including fixed-base serial arms, parallel and Cartesian mechanisms, grippers and dexterous hands, wheeled and tracked bases, legged robots from biped to hexapod, a serpentine chain, and an aerial vehicle. They also vary in how the target shape is specified. Twenty are given in text alone; five are *image-conditioned* (Boston Dynamics Spot, an industrial arm, WALL-E, an Open Duck Mini-style bipedal duck, and a servo-driven humanoid), each supplying a reference photograph the model is expected to match in structure and proportion. They vary likewise in geometric demand: many morphologies are adequately represented by boxes, cylinders and capsules, while others, such as mecanum rollers, continuous treads, propeller blades, and a multi-fingered palm *etc.*, have no primitive equivalent and require custom mesh geometry to be represented faithfully. Each prompt carries a compliance specification encoding only what the described robot truly demands, such as a floating base or a minimum number of hinge joints and actuators.

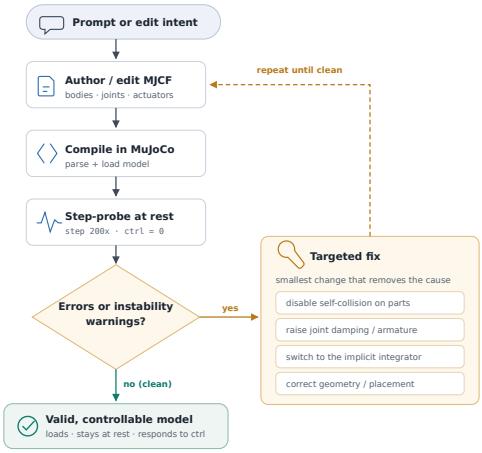


Fig. 3. The self-verifying loop. After every edit the model is compiled and step-probed at rest (200 steps at zero control); any parse error or instability warning (deep interpenetration / runaway velocity) is returned to the agent, which fixes it—most often by disabling self-collision among the robot's own link geoms, adding damping/armature, switching to an implicit integrator, or adjusting geometry—and the cycle repeats until the model is clean.

C. Baseline comparison

We compare the shipped system against a naive use of the same LLM, generating every prompt under matched conditions and scoring all outputs identically:

- **Direct LLM (baseline):** the same LLM on the same task in the same kind of isolated session, but with a neutral prompt and none of our domain tools. It may author and edit MJCF and check whether the model loads, and it sees the parser errors that check returns, but it receives no physics guidance and no verification feedback. Having no `build_part` tool it can only compose primitive geometry, an honest property of the baseline rather than an imposed handicap.
- **No verifier:** our agent with its domain prompt and its complete authoring toolset, with `validate` included, but with both verification signals switched off. `validate` still compiles the model and returns parse errors, and `build_part` still produces a mesh, yet neither the rest-pose stability probe nor the CAD structural check runs, so the agent never receives a physics or geometry warning. Against the baseline this isolates the contribution of domain knowledge and the authoring tools; against the full loop it isolates verification alone, with the system prompt, the tool set, and the compile feedback all held fixed.
- **Full loop (ours):** `validate` present with the rest-pose stability probe enabled, the shipped system.

Generation is otherwise identical across conditions: the same LLM, the same prompt text, the same reference image where a prompt is image-conditioned, and the same number of seeds per prompt. Each generation runs in a fresh isolated session with no knowledge of the other conditions, and every output is graded post hoc by the same independent scorer, which re-compiles and re-probes the final model rather than trusting any verdict the agent reached while authoring. The baseline and the full loop differ along several axes at once, including system prompt, tool availability, and verification signal, so the difference between them measures the effect of the system as a whole rather than of any single mechanism. The intermediate condition is what separates those axes: it shares the full system's prompt and tools, and differs from it only in whether verification is available. Conditions are compared pairwise over the same prompts, so we report Wilson intervals on each rate.

TABLE I
INDEPENDENT SCORER ON ONE MODEL PER BENCHMARK PROMPT ($n = 25$). PASS = STABLE $\wedge$ CONTROLLABLE $\wedge$ REQUIREMENT-COMPLIANT; COMPILATION IS IMPLIED.

Metric	Value
Generations scored	25
Invalidity rate (IR)	0.00
pass@1	0.92
Stable (rest-pose probe)	1.00
Controllable (drive probe)	1.00
Requirement compliance	0.92

D. Automatic vision form judge

Physics checks cannot see that a wrist is rotated ninety degrees, a leg is flipped, or a chassis is placed asymmetrically. We therefore render the compiled model at its rest pose from four canonical viewpoints (Fig. 5: a head-on view, two symmetric 45° three-quarter views, and a top view), each auto-framed on the model's bounding sphere, and pass the renders to a vision model that returns a structured verdict: overall pass/warn/fail, per-criterion booleans (upright, symmetric, parts-oriented), a list of issues, and, when the model was image-conditioned, whether it matches the reference image. The judge is *report-only*: it never affects the pass/fail score, so the physics metrics remain comparable to prior work. Because the judge is sampled and therefore non-deterministic, reported numbers use a k-sample majority vote per criterion, and we calibrate the judge against human ratings (Section V-D).

V. RESULTS

A. Benchmark model validity

We scored one model for each of the 25 benchmark prompts with the independent scorer (Table I). Every model compiles, so the invalidity rate is 0.00; all 25 are stable under the rest-pose probe and all 25 are controllable. Twenty-three of the 25 satisfy their requirement specification in full (compliance rate 0.92), and averaged over models, a mean 0.97 of the individual requirements are met. Both failures are humanoids, one text-specified and one image-conditioned, authored on a fixed base although their specifications demand a floating one. Each still meets its joint- and actuator-count requirements. pass@1 is therefore 0.92, bounded entirely by requirement compliance rather than by physics.

TABLE II

LOOP ABLATION OVER ALL 25 BENCHMARK PROMPTS, ONE GENERATION PER CELL ($n = 25$ PER CONDITION). ALL THREE CONDITIONS USE THE SAME LLM, THE SAME PROMPT TEXT, AND THE SAME REFERENCE IMAGES; EVERY OUTPUT IS GRADED POST HOC BY THE SAME INDEPENDENT SCORER. INVALIDITY RATE IS 0.00 IN ALL THREE AND IS OMITTED. THE *stable* COLUMN IS MEASURED BY THE SAME REST-POSE PROBE THE FULL-LOOP AGENT ITERATES AGAINST; THE TRIM COLUMNS ARE HELD OUT FROM EVERY CONDITION.

Condition	Scored				Held out	
	pass@1	stable	ctrl	reqs	trim ≤ 1 cm	drop (mm)
Direct LLM	0.32	0.40	0.96	0.84	0.24	70.6
No verifier	0.64	0.72	1.00	0.84	0.48	12.9
Full loop (ours)	0.92	1.00	1.00	0.92	0.92	0.2

Because the scorer's stability check and the agent's own verifier invoke the same rest-pose probe, that check is close to true by construction for this condition. We therefore also applied a *held-out* trim check that the agent never sees in any condition: each model is released under gravity at zero control and stepped until its velocities decay, and we record how far its base travels. Twenty-three of the 25 settle within 1 cm of their authored pose. The two exceptions—the four-wheel rover (5.0 cm) and the image-conditioned Spot(2.3 cm)—spawn with no ground contact and drop onto the floor before coming to rest, so they are mis-placed at authoring time rather than unstable once settled. These figures characterize the models the system delivers, not the marginal contribution of the authoring loop; the latter is isolated in the ablation below. They are measured in the system's intended operating mode. Prompt2Kinematics is an interactive authoring tool, not a one-shot generator: the designer is expected to play the model, inspect it, and refine it by further instruction or direct manipulation, and the self-verifying loop runs on every one of those edits, not only on the first. What the table reports is the quality of the artifact a designer ends up with, which is the quantity the system is built to maximize. The ablation below holds the number of turns fixed instead, because isolating the verifier's contribution requires the conditions to differ in one respect only.

B. Effect of the self-verifying loop

Table II shows a monotonic ladder in pass@1: 0.32 for a direct use of the same LLM, 0.64 once the agent is given our domain prompt and authoring tools, and 0.92 with verification enabled. Invalidity is 0.00 in all three conditions, so compilation success separates none of them—a model that parses is not a model that stands. Domain knowledge and verification contribute comparable amounts, roughly $+0.32$ and $+0.28$ pass@1, and neither alone accounts for the system's behavior.

What verification actually catches. Inspecting the independent probe's warnings shows the failures are overwhelmingly static, not dynamic: of the direct LLM's 15 unstable models, 14 interpenetrate at the rest pose and only one diverges under integration. These split into two distinct authoring errors: the robot's own link geoms overlapping at a joint and the whole body authored below the ground plane. The no-verifier condition reduces this to 7 models, and the full loop eliminates it. The defect, in other words, is present before a single step is taken and is invisible to the compiler, which is precisely the gap the step-probe closes and the reason the invalidity rate discriminates nothing.

A non-circular check agrees. The *stable* column is measured by the same rest-pose probe the full-loop agent iterates against, so that condition is trained on the metric. We therefore applied the held-out trim check of Section V-A—release under gravity at zero control, step until velocities decay, record base displacement—to all three conditions; no agent receives this signal in any of them. The ladder is preserved: 0.24, 0.48 and 0.92 of models settle within 1 cm, against the circular stability rates of 0.40, 0.72 and 1.00. The continuous form is sharper still, with median base displacement falling from 70.6 mm for the direct LLM to 12.9 mm without verification and 0.2 mm with it. Because no condition is optimized against this measure, it corroborates the stability column rather than restating it.

The comparison between the no-verifier and full-loop conditions isolates the effect of verification: the domain prompt, the authoring tools, and the compile feedback are shared between them, and they differ only in whether the rest-pose probe and the CAD structural check run. On the 11 prompts where all three conditions complete in a single authoring turn, the ordering is unchanged—0.45, 0.73 and 1.00 pass@1—so the ladder is not an artifact of the full loop taking more turns.

Validity is not fidelity. The scorer measures whether a model is physically sound, not whether it visually or structurally resembles what was requested. The conditions differ substantially on this dimension in ways that are not captured by any column of Table II. None of the direct LLM's 25 models contains a mesh geom because, without `build_part`, the baseline can only compose primitive geometry. In contrast, 9 of 25 models produced by the full loop contain custom meshes. The full loop also averages 23.2 geoms per model compared with 17.4 for the baseline, while maintaining an identical mean actuator count. A quadruped constructed from seven boxes and one constructed from custom CAD shells may therefore receive the same validity score even though their geometric fidelity differs substantially. This motivates the use of the vision judge below as a complementary measure of form and reference consistency.

C. Geometric and pose correctness

We applied the vision form judge to all generated models, which evaluated basic properties such as uprightness, symmetry, and part orientation. When comparing the models with their reference photos, the judge correctly identified structural similarities (matches reference = true) while also noting minor morphological differences from the source

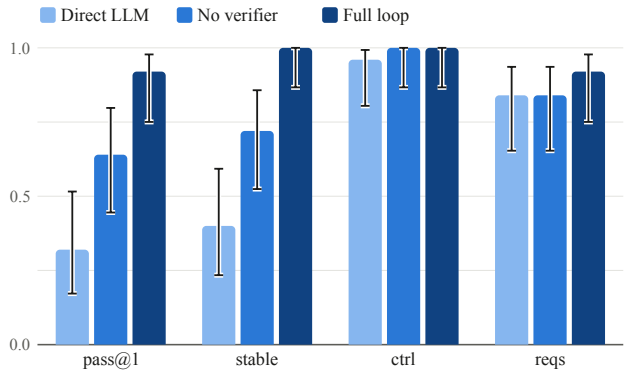


Fig. 4. The loop ablation of Table II. Bars are rates over the 25 benchmark prompts; whiskers are Wilson 95% intervals. The gain is carried by pass@1 and stability—controllability is already near ceiling, and requirement compliance, which the loop never receives feedback on, barely moves.

images. Furthermore, the judge discriminates effectively in the negative direction: when applied to the naive baseline models (the Direct LLM row of Table II), it correctly flagged major structural failures—such as missing appendages or absent joints—returning matches reference as false, even while accurately recognizing those same models as upright and symmetric. This illustrates that the vision judge catches reference-relative details that a purely physics-based scorer inherently cannot. However, the judge is not infallible. Reasoning only from rest-pose renders, it occasionally missed specific functional defects (such as incorrectly oriented locomotive parts) that a human expert would spot immediately. Because of these blind spots, we treat its verdicts as report-only and calibrate them against human ratings (Section V-D).

D. Judge calibration

Three blinded experts rated 25 models: unanimous on 24/25, split 2–1 on one. Fleiss' $\kappa = 0.79$ (Krippendorff's $\alpha = 0.79$; pairwise Cohen's κ 1.00, 0.65, 0.65). With 93% Pass ratings, κ is deflated by low marginal variance (the single split alone drops it from 1.00 to 0.79); Gwet's $AC_1 = 0.97$ and Brennan–Prediger $= 0.95$ indicate substantial consistency. Against the majority label (2 Fail, 23 Pass), the judge agreed on 23/25 (92%, Wilson 95% CI $[75\%, 98\%]$; $\kappa = 0.63$), catching both consensus-Fail models but flagging two Pass models. It is thus more conservative than the panel—preferable for a screen, making Pass@1 a lower bound. With only two Fail models, though, these figures are a sanity check, not a precise estimate; a larger audit is needed.

E. Cost of verification

Verification is not free, but it is cheap relative to what it buys. Over the single-turn prompts, the full loop takes a median 94 s and 168 k billed tokens per model, against 55 s and 135 k for the direct LLM—roughly +70% wall-clock and +24% tokens to move pass@1 from 0.32 to 0.92. Approximately 87% of billed tokens are cache reads;

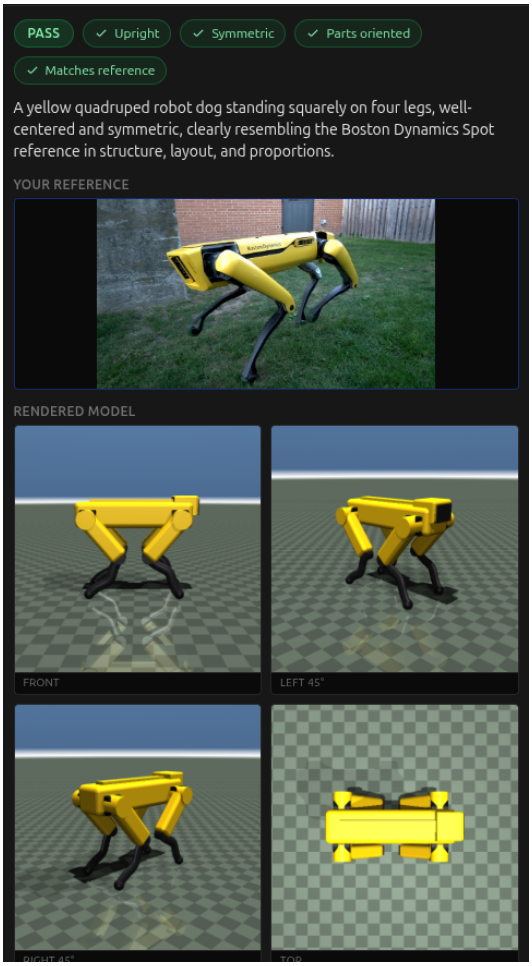


Fig. 5. The form-check report for the image-conditioned Spot model. Top: the judge's verdict card with four criteria and summary. Middle: the reference photograph. Bottom: four canonical rest-pose renders (front, left 45°, right 45°, top). The verdict is report-only and does not affect the score.

the median full-loop run consumes 144 k cache-read, 16.7 k cache-write, and 6.5 k output tokens on 10 fresh input tokens, so the marginal cost of verification is far below the billed total. The no-verifier condition is the outlier at 689 k median tokens, with one run reaching 2208 s: instructed to author through tools whose verification path is disabled, the agent retries against signals it never receives.

In the interactive mode the system is built for, a model takes a median 7.5 min of active generation time (range 1.8–15.9 min) over a median 3 user turns (1–9) and 14 tool calls (3–58), or about 2.9 min per turn, at a median 940 K billed tokens. The gap between this and the single-turn figures is the designer's own iteration, not verifier overhead: the loop runs on every edit, so a nine-turn session pays for verification nine times over.

VI. Limitations

Several limitations bound the current results. First, every cell is a single generation ($k = 1$), so pass@k collapses onto pass@1 and we characterize neither run-to-run variance

nor its effect on the reported deltas. The two prompts the no-verifier condition loses relative to the baseline are within the noise that a single sample cannot separate from a real regression. Second, the *stable* rate is not fully independent of the full-loop condition: the scorer and the agent's `validate` tool invoke the same rest-pose probe, so that condition is iterated until exactly the quantity being scored comes back clean. We report it because the ablation's other conditions are scored by the identical procedure without ever seeing it, and we lean on the held-out checks, including controllability, requirement compliance, and the trim check of Section V-A, where a circularity objection does not apply. However, the stability column should be read as the loop succeeding at its stated objective rather than as independent corroboration. Third, the vision form judge is an LLM and is non-deterministic. Although majority voting reduces variance and we report human agreement, it is a heuristic ratherthan ground truth, and it reasons from four rendered views at a single rest pose. Defects visible only under motion or from other angles can therefore be missed. Fourth, the requirement-compliance check is structural (counts of joints, actuators, and a floating base) and does not verify that a control behaves as a user would expect. Agent-authored behavior tests would be circular and are deliberately excluded from scoring. Finally, our evaluation measures the produced artifact; a study of the interactive authoring experience, including grounded direct manipulation and incremental repair, is future work.

VII. Conclusion

Prompt2Kinematics turns a natural-language prompt, conditioned on a reference image, into a valid, controllable MuJoCo model by placing an LLM agent inside a self-verifying loop that compiles and step-probes every edit, and by making the resulting model a live simulation the designer edits through grounded direct manipulation. An independent scorer and a report-only vision judge let us quantify both physical validity and geometric correctness. Across a 25-prompt benchmark the system delivers models that are stable and controllable in every case, with pass@$1 = 0.92$ bounded by structural requirements rather than by physics. A three-condition ablation over the same prompts separates where that comes from: a direct use of the same LLM passes 0.32, adding our domain prompt and authoring tools raises it to 0.64, and enabling verification raises it to 0.92. The failures verification removes are almost entirely static, geometry that interpenetrates before the model is ever stepped, which is exactly the class of defect that compiles cleanly and that a designer would otherwise discover only by running the model.

VIII. AI Disclosure

In accordance with the IEEE Robotics and Automation Society (IEEE-RAS) policies on the use of Artificial Intelligence (AI) and AI-generated content, the authors disclose the following usage of AI tools during the preparation of this manuscript:

- **Text Editing and Polishing:** Large Language Models (*e.g.*, ChatGPT) were utilized strictly for copyediting, correcting grammatical errors, and polishing the final text to improve overall readability.
- **Coding Assistance:** AI-assisted coding tools (*e.g.*, Claude Code) were employed to help write boilerplate code, structure data parsing, and troubleshoot standard programming logic used in the implementation of our experiments.
- **Figure and Diagram Generation:** Generative AI and AI-assisted design tools were used to refine the visual presentation and improve the layout of the workflow diagrams presented in this paper.

The specific AI tools were used solely as assistive technologies. The authors have rigorously reviewed all AI-assisted outputs and assume full accountability and responsibility for the integrity, accuracy, and validity of the final manuscript, experimental code, and figures.

References

[1] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.

[2] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal, "Text2CAD: Generating sequential CAD designs from beginner-to-expert level text prompts," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[3] L. Le, J. Xie, W. Liang, H.-J. Wang, Y. Yang, Y. J. Ma, K. Vedder, A. Krishna, D. Jayaraman, and E. Eaton, "Articulate-Anything: Automatic modeling of articulated objects via a vision-language foundation model," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.

[4] Z. Li, X. Bai, J. Zhang, Z. Wu, C. Xu, Y. Li, C. Hou, and S. Zhang, "URDF-Anything: Constructing articulated objects with 3D multimodal language model," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[5] R. P. Ringel, Z. S. Charlick, J. Liu, B. Xia, and B. Chen, "Text2Robot: Evolutionary robot design from text descriptions," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2025, doi: 10.1109/ICRA55743.2025.11128168.

[6] K. Qiu, W. Pałucki, K. Ciebiera, P. Fijałkowski, M. Cygan, and Ł. Kuciński, "RoboMorph: Evolving robot morphology using large language models," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2026.

[7] L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang, "GenSim: Generating robotic simulation tasks via large language models," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[8] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, vol. 235, pp. 51936–51983, 2024.

[9] B. W. Lim, M. D. Le, T. Truong, and T. N. Canh, "SR-Platform: An agentic pipeline for natural language-driven robot simulation environment synthesis," arXiv preprint arXiv:2605.14700, 2026.

[10] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2024.

[11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[12] B. Shneiderman, "Direct manipulation: A step beyond programming languages," *Computer*, vol. 16, no. 8, pp. 57–69, 1983.

[13] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.